



CLOUD DATA WAREHOUSING

COLLÈGE INGÉNIERIE ET ARCHITECTURE

SEMESTRE 8

Pr. Anouar Riad Solh
v-anouar.riadsolh@uir.ac.ma
 2025/2026

1

Plan:

Introduction aux bases de données distribuées

Stockage de données distribué :

Réplication

Fragmentation : horizontale, verticale

Autorisation et contrôle d'intégrité

Traitement et optimisation des requêtes distribuées

Gestion des transactions distribuées

Contrôle de concurrence distribué et contrôle de réplication

Récupération après incident et haute disponibilité

Bases de données NoSQL distribuées

Présentation des SBD distribuées dans les fournisseurs de cloud public (Azure et AWS)

Introduction à Data Warehousing:

Concepts principaux

Cas d'utilisation et avantages

Extraire, Transformer, Charger (ETL)

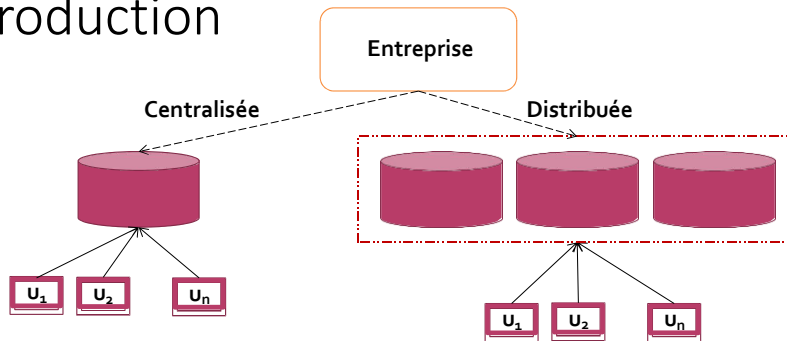
Datamart

Architecture de l'entrepôt de données

Entrepôts de données distribués

2

Introduction

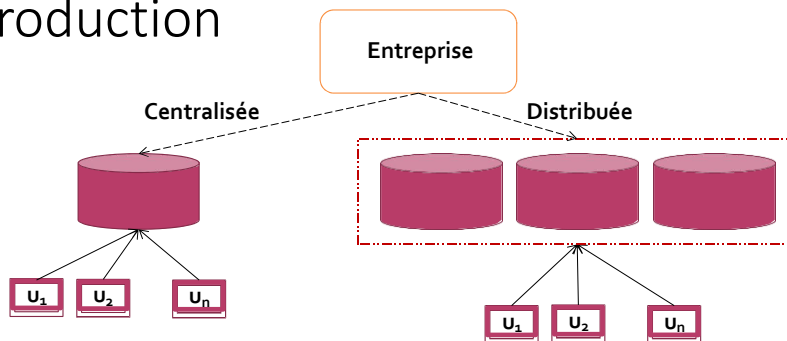


Problématique:

- une entreprise est installée au Maroc
 - Ses filiales se trouvent dans 3 régions
 - Chaque filiale gère les clients de sa région
 - Chaque filiale peut accéder aux informations de la centrale OU/ET aux informations des autres filiales .
- Quelle est l'architecture proposée

3

Introduction



Une **base de données distribuée** est une base de données dont les éléments de données **sont répartis sur plusieurs sites, régions ou même différents pays**. Contrairement aux bases de données traditionnelles qui sont centralisées, les bases de données distribuées tirent parti de **l'architecture en réseau pour connecter les bases de données physiques séparées en une seule base de données logique**.

4

Introduction

• Problème de base de données centralisées



- L'augmentation du volume de données
- L'augmentation du volume de transactions.
- L'augmentation du temps de traitement

Multiplier les Base , et les fait communiquer par des réseaux

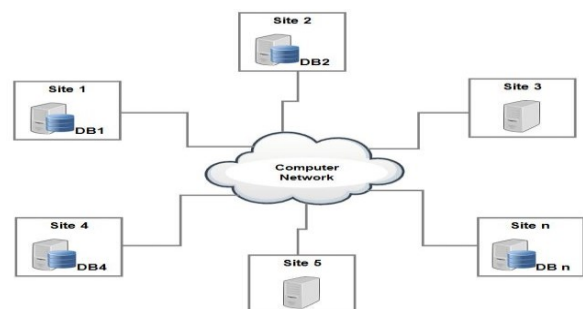


Base de données distribuées

5

Qu'est ce qu'une base de données distribuée?

- Une base de données distribuée est **répartie et gérée par des sites différents** et **apparaissant à l'utilisateur comme une base unique**.
- Une collection de multiple bases de données **distribuées sur un réseau informatique** logiquement interreliées.



6

Qu'est ce qu'une base de données distribuée?

Un **client** d'un système de gestion de base de données distribuée

- Une **application** qui accède aux informations distribuées par les interfaces du système (Exemple: un navigateur Web).

Un système de gestion de bases de données distribuées (**SGBD**)

- l'application qui permet la gestion de la base de données distribuée et rend la distribution transparente aux utilisateurs

7

Les avantages des bases de données distribuées

- ❑ **Continuité de service** : en cas de panne d'un serveur, un autre **prend la relève**.
- ❑ **Haute performance** : les requêtes sont **distribuées** sur les différents serveurs.
- ❑ **La transparence des données** : Les développeurs et les utilisateurs **n'ont pas à se soucier** de l'emplacement géographique des données qu'ils utilisent.

8

Objectifs d’une base de données distribuées

Exemple

- Soit une entreprise qui a des annexes dans plusieurs villes: Rabat, Casablanca et Tanger.
- Le responsable veut maintenir une base de données des employés et les projets sur lesquels ils travaillent.
- Nous avons 4 tables:
Emp (idEmp, nomEmp, poste);
Prj (idPrj, nomPrj, budget, ville);
Sal (poste, salaire);
Aff (idEmp, idPrj, resp, dur)
//Affectation d’un employé à un projet avec telle responsabilité pendant une durée.

9

Exemple

Emp

idEmp	nomEmp	poste
E1	Sm	Comptable
E2	Md	Ing.
E3	Hm	Elec.Ing
E4	Ad	Programmeur

Aff

idEmp	idPrj	resp	dur
E1	P1	Directeur	12
E2	P1	Analyste	24
E2	P2	Analyste	6
E3	P3	Consultant	10
E3	P4	Ingénieur	48
E4	P2	Programmeur	18

Prj

idPrj	nomPrj	budget	ville
P1	Instrumentation	15000	Rabat
P2	BDD develop.	135000	Casablanca
P3	Construction	250000	Casablanca
P4	Entretien	310000	Tanger

Sal

poste	salaire
Comptable	20000
Ing.	34000
Elec.Ing	27000
Programmeur	24000

10

Objectifs d'une base de données distribuées

Exemple

Si la base de données est **centralisée**, on peut trouver les noms des employés qui travaillent dans un projet pour une durée supérieure de 12 mois ainsi que leurs salaires avec la requête suivante:

11

Objectifs d'une base de données distribuées

Exemple

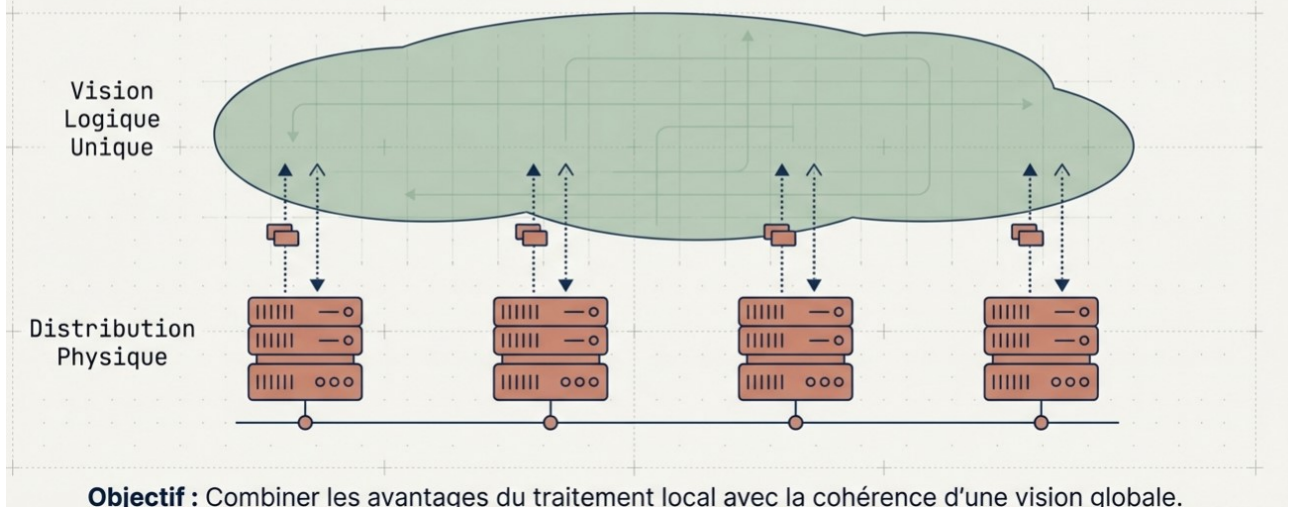
Si la base de données est **centralisée**, on peut trouver les **noms des employés qui travaillent dans un projet pour une durée supérieure de 12 mois** ainsi que leurs salaires avec la requête suivante:

```
SELECT nomEmp, salaire
FROM Emp, Aff, Sal
WHERE Aff.dur > 12
AND Emp.idEmp = Aff.idEmp
AND Sal.poste = Emp.poste
```

12

La Solution : La Base de Données Répartie

Une **collection** de données logiquement reliées, mais physiquement distribuées sur un réseau informatique.



13

Objectifs d'une base de données distribuées

Exemple

Maintenant, si la base est distribuée sur les **différents sites**, de telle sorte que les informations des employés et les projets de chaque site sont stockés dans **une base de données localisée dans ce site**.

L'utilisateur doit poser la même question au système **sans faire attention à la distribution des informations(transparence)**, et c'est **le système qui s'occupe de la récupération des informations**.

Pour cela, il a été défini plusieurs types de transparence

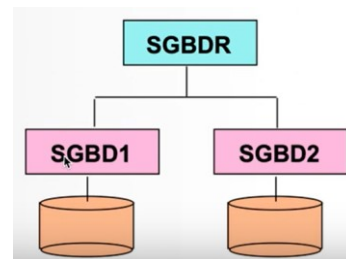
14

Objectifs d'une base de données distribuées

- ❖ **Rendre la Distribution transparente:** **logiquement nous avons une seule BD mais physiquement nous avons plusieurs BD réparties**

Pour installer BDR, il faut un SGBDR comme oracle (rendre les données disponibles par tout et assure la transparence):

- **Dictionnaire de données réparties**
- **Traitement des requêtes réparties**
- **Gestion des transactions réparties**
- **Gestion de la cohérence et de la sécurité**



on peut installer un seul SGBDR centralisé dans le site central et installer aussi un SGBD dans chaque site pour gérer la BD Localement dans chaque site

15

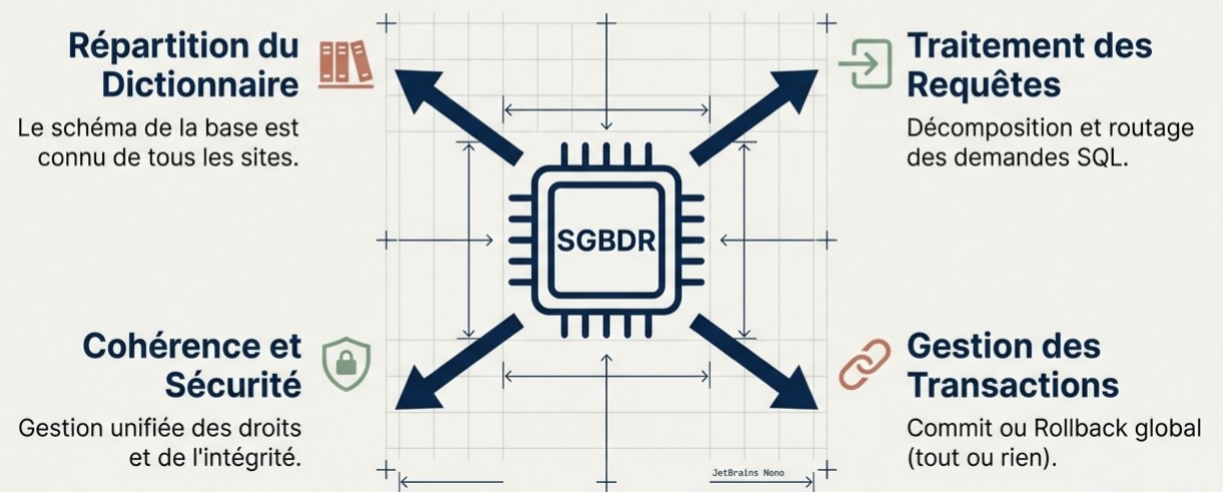
Distribution

- ❖ **Meilleures performances:**
 - **Réduire le trafic** sur le réseau est une possibilité d'accroître **les performances**.
 - **Répartir la charge sur les processeurs** et sur les entrées/ sorties.
- ❖ **Faciliter l'accroissement(extention):** l'accroissement se fait **par l'ajout de machines** sur le réseau.
- ❖ **Disponibilité** en raison de l'existence de **plusieurs copies**.
- ❖ **Maintient d'une vision unique de la base de données malgré la répartition.**

16

Le Moteur : SGBD Réparti

L'orchestration par le logiciel (ex : Oracle)



17

Pourquoi Répartir ? Les Objectifs Clés



18

Principe fondamentale des DBR

➤ **Autonomie locale**

La BD locale est complète et autonome (intégrité, sécurité, gestion), elle peut évoluer indépendamment des autres (upgrades...)

➤ **Egalité entre sites**

Un site en panne ne doit pas empêcher le fonctionnement des autres sites (mais perturbations possibles)

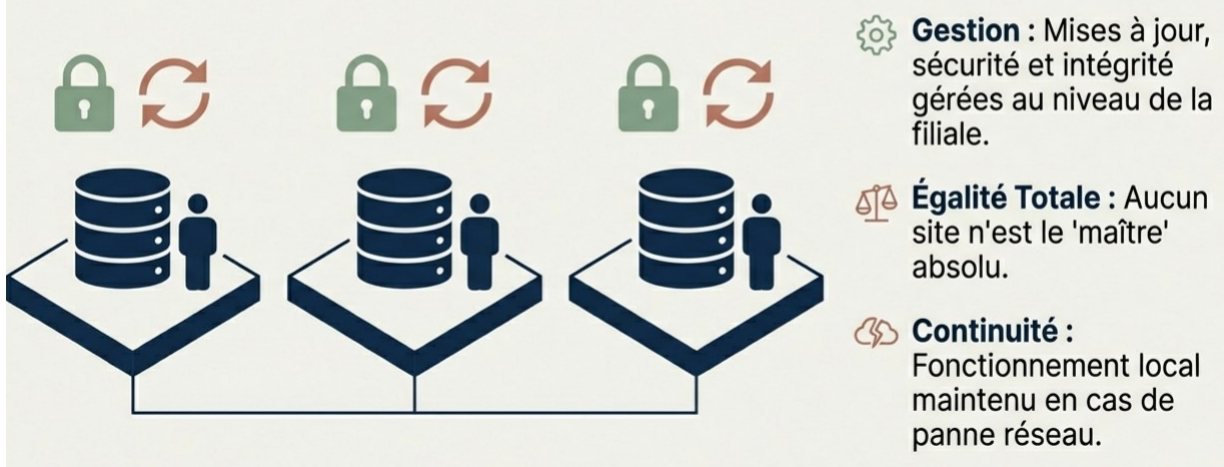
➤ **Fonctionnement continu**

Distribution permet résistance aux fautes et aux pannes

19

Principe 1 : L'Autonomie Locale

Chaque site est maître de son destin, mais partie d'un tout.



20

Principe fondamentale des DBR

➤ Localisation transparente

Accès uniforme aux données quel que soit leur site de stockage

➤ Fragmentation transparente

Des données (d'une même table) éparpillées doivent être vues comme un tout

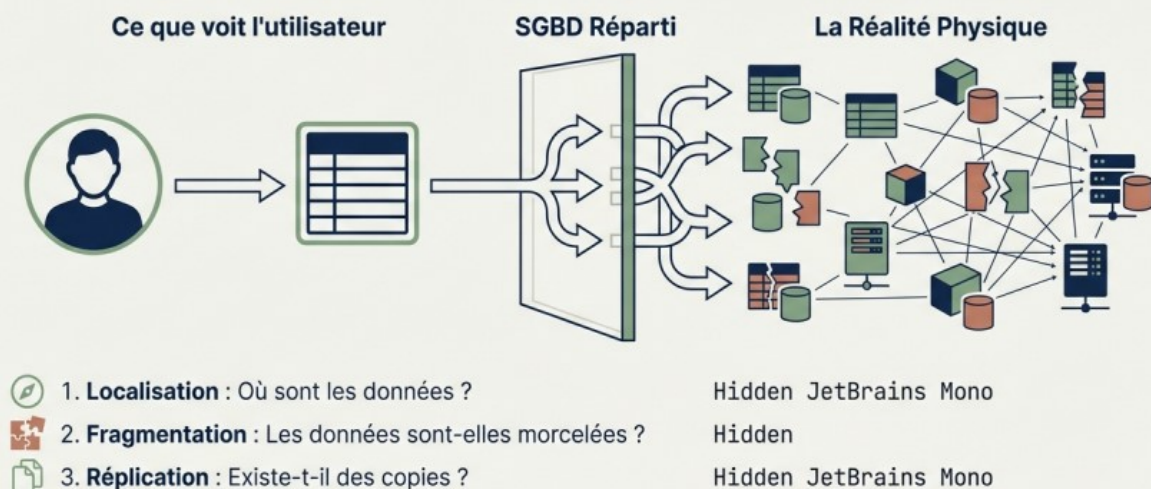
➤ Indépendance à la réplication

Les données répliquées doivent être maintenues en cohérence

21

Principe 2 : La Transparence

L'illusion de l'unicité pour l'utilisateur



22

Principe fondamentale des DBR

➤ Requêtes distribuées

L'exécution d'une requête peut être répartie (automatiquement) entre plusieurs sites (si les données sont réparties)

➤ Transactions réparties

Le mécanisme de transactions peut être réparti entre plusieurs sites

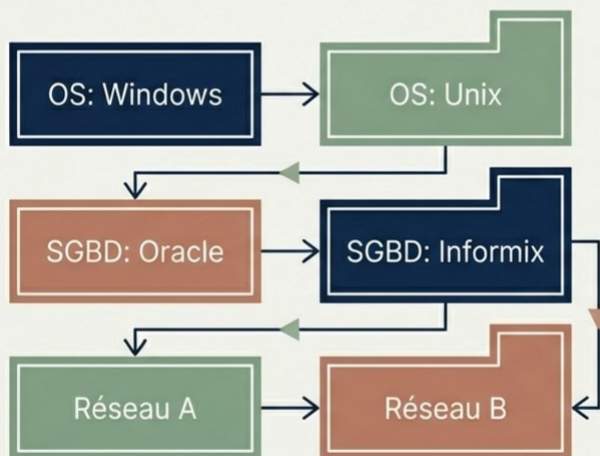
➤ Indépendance vis-à-vis du matériel

Le SGBD fonctionne sur les différentes plateformes utilisées

23

Principe 3 : Indépendance Technique

Hétérogénéité des systèmes



Indépendance Logique :

Une seule requête peut toucher plusieurs sites.



Indépendance Physique :

Mixité du matériel et des OS.



Interopérabilité :

Communication entre différents vendeurs de bases de données.

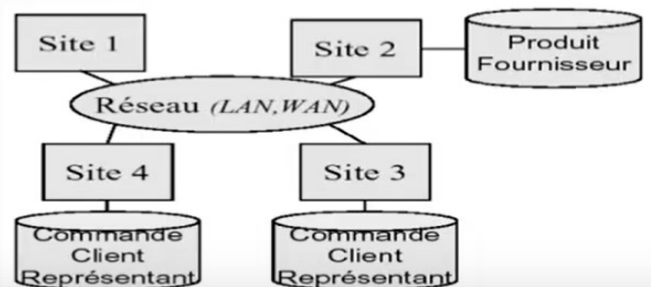
24

Exemple : Base de données distribuée

Schéma global :

- ❖ - **Produit** (NoProd, DesProd, Prix, NoFour)
- ❖ - **Fournisseur** (NoFour, NomFour, VilleFour)
- ❖ - **Client** (NoCli, NomCli, VilleCli)
- ❖ - **Représentants** (NoRep, NomRep, VilleRep)
- ❖ - **Commande** (NoProd, NoCli, Date, Qte, NoRep)

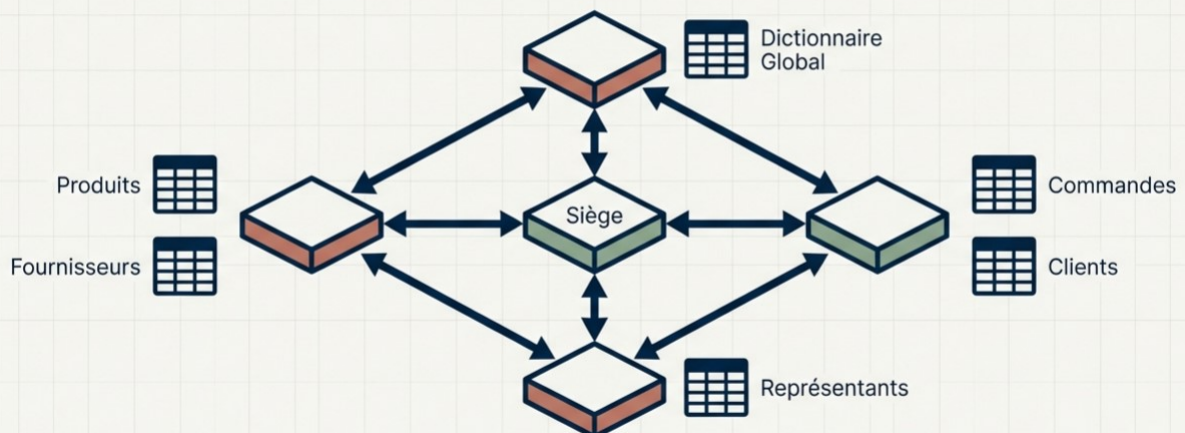
Schéma des données réparties :



25

Mise en Œuvre : Exemple de Schéma Global

Comment répartir les tables d'une entreprise ?



Résultat : Chaque site héberge une partie du schéma, l'ensemble forme la base cohérente.

26

Approche BDR

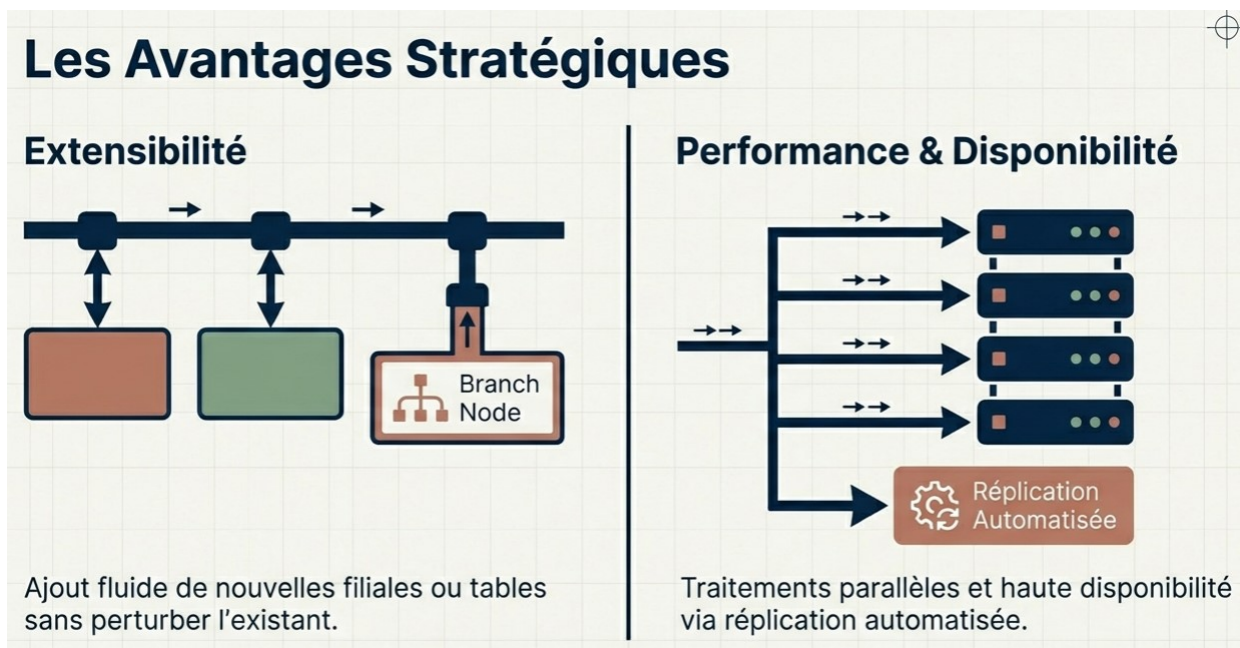
☐ avantages

- ☐ extensibilité
- ☐ partage des données hétérogènes et réparties
- ☐ performances avec le parallélisme
- ☐ Disponibilité et localité avec la réplication

☐ inconvénients

- ☐ administration complexe
- ☐ distribution du contrôle
- ☐ surcharge (l'échange de messages augmente le temps de calcul)

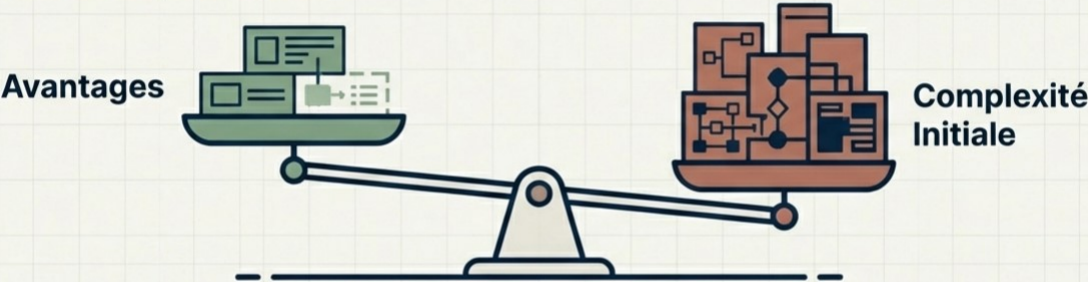
27



28

Les Défis et Inconvénients

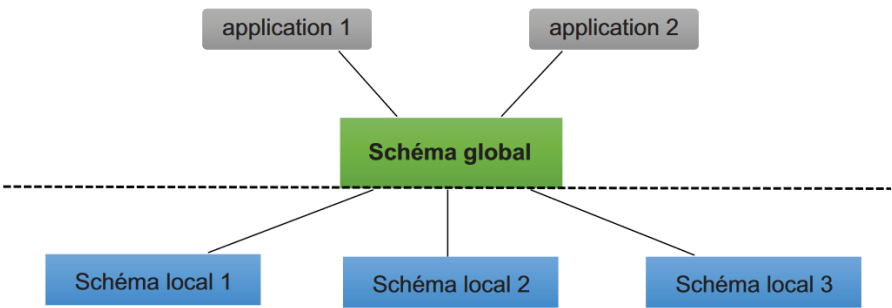
Le coût de la puissance



- ⚠ **Complexité d'Administration** : Lourd au démarrage (définition de la fragmentation).
- ⚠ **Investissement** : Temps de configuration initial important.
- ⚠ **Migration** : Passer du centralisé au réparti est risqué pour l'historique des données.

29

Architecture BDR



- indépendance applications / bases locales
- schéma global lourd à gérer

30

Schéma global

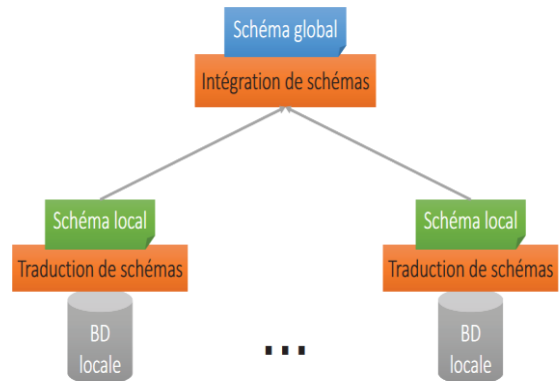
❑ Schéma conceptuel global

- description globale et unifiée de toutes les données de la BD Distribuées
- Nom des relations avec leurs attributs
- Fournir l'indépendance à la répartition

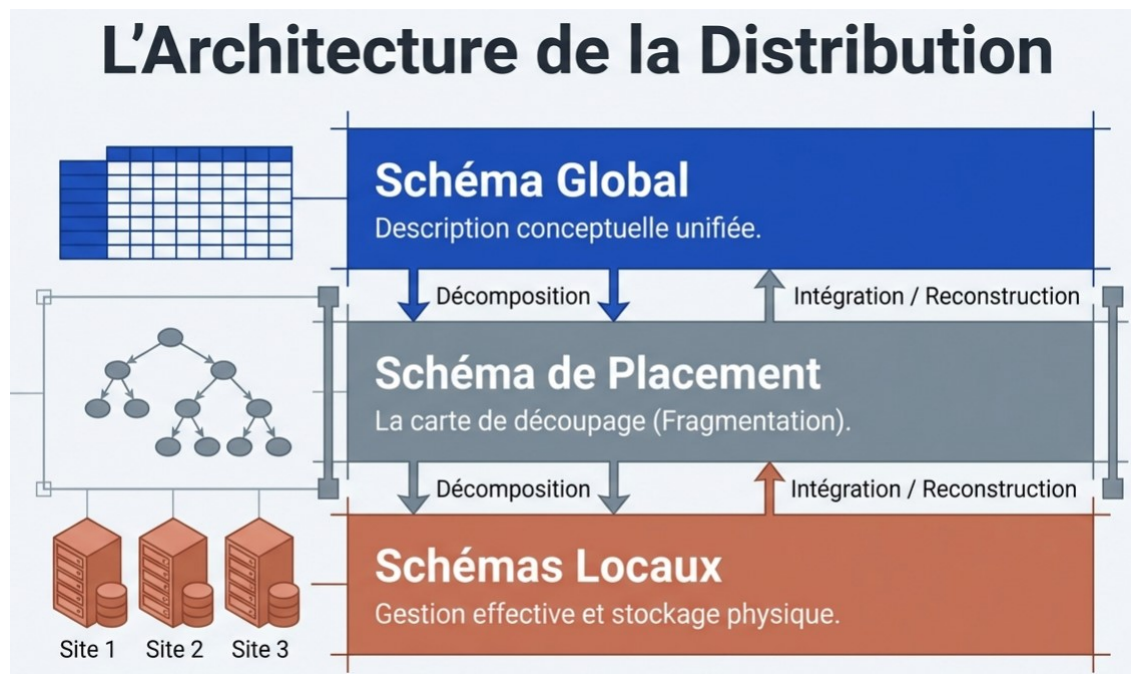
❑ Schéma de placement

- règles de correspondance avec les données locales
- Vues globales définies sur les relations locales
- Fournit l'indépendance à la localisation, la fragmentation et la duplication

❑ Le schéma global fait partie du dictionnaire de la BDD et peut être conçu comme une BDD (duplicqué ou fragmenté)



31



32

Schéma global

Schéma conceptuel global

Client (nclient, nom, ville)

Cde (ncde, nclient, produit, qté)

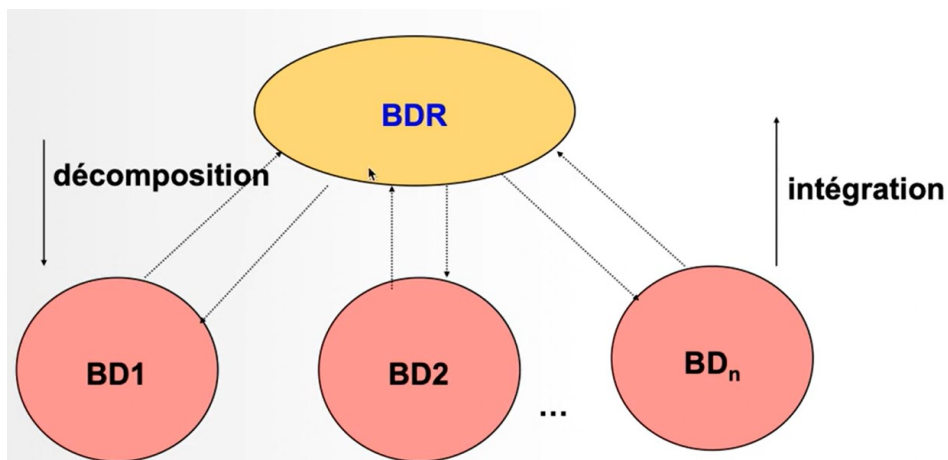
Schéma de placement

Client = Client1 @ Site1 U Client1 @ Site2

Cde = Cde @ Site3

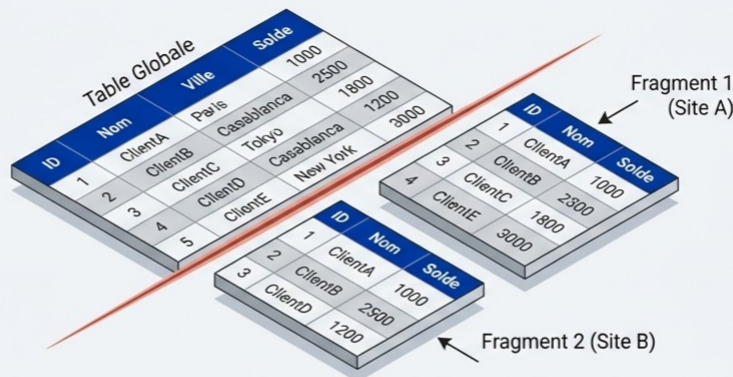
33

Conception BDR : 2 approches



34

La Stratégie de Décomposition



Concept : La fragmentation consiste à diviser une table globale en plusieurs sous-ensembles logiques.

Motivation : Optimisation de la localité. Placer les données là où elles sont le plus utilisées.

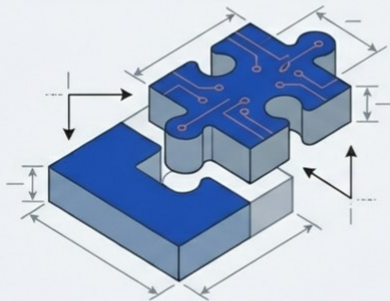


Exemple : Les données des clients résidant à Casablanca sont stockées physiquement sur le serveur de l'agence de Casablanca pour réduire le trafic réseau.

35

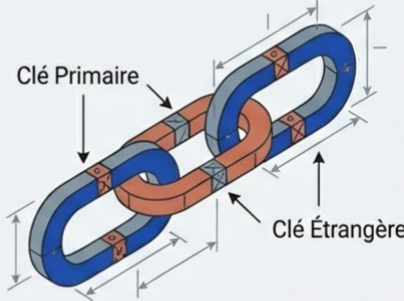
Les Règles d'Or de l'Intégrité

1. Préservation des Données



Décomposition sans perte (Lossless). Il doit être possible de reconstruire la relation originale exactement à partir des fragments. Aucune donnée ne doit être perdue ou créée lors de la recombinaison.

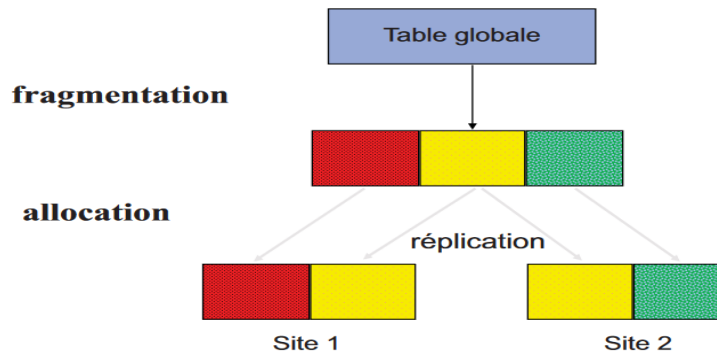
2. Intégrité des Contraintes



Chaque fragment doit respecter les contraintes du schéma global, notamment les Clés Primaires et les Clés Étrangères. Une insertion locale doit garantir la cohérence globale.

36

Conception par décomposition



Réplication

La réplication consiste à **dupliquer les données sur plusieurs nœuds**. Chaque nœud peut contenir une copie complète ou partielle de la base de données.

37

Fragmentation : horizontale, verticale

Objectifs :

- **Améliorer les performances** : En stockant les données proches des utilisateurs qui y accèdent fréquemment.
- **Réduire la charge** : En répartissant les données et les requêtes sur plusieurs nœuds.
- **Scalabilité** : Permet d'ajouter plus de nœuds pour gérer des volumes de données plus importants.

38

Fragmentation

La fragmentation est le processus de **décomposition d'une BD centrale** en un ensemble de sous bases de données.

Les types de fragmentation :

- Fragmentation horizontale
- Fragmentation verticale
- Fragmentation Mixte

39

FRAGMENTATION HORIZONTALE



Mécanisme : Découpage par LIGNES (Tuples).



Opérateur : Sélection (σ).



Reconstruction : UNION (U).

CLIENT		
nclient	nom	ville
C1	Alami	Casa
C2	Saoudi	Rabat
C3	Saber	Fès
C4	Anouari	Casa

Client1 (Casa)		
C1	Alami	Casa
C4	Anouari	Casa

Client2 (Autres)		
C2	Saoudi	Rabat
C3	Saber	Fès

Client1 = $\sigma[\text{ville} = \text{'Casa'}]$ Client
Client = Client1 U Client2

40

Fragmentation horizontale

Fragments définis par sélection

- Client1 = Client where ville = "Casa"
- Client2 = Client where ville ≠ "Casa"
- Client1 = σ [ville = 'casa'] Client
- Client2 = σ [ville <> 'casa'] Client

Reconstruction

Client = Client1 U Client2

Client

nclient	nom	ville
C 1	alami	Casa
C 2	saoudi	Rabat
C 3	saber	Fès
C 4	anouari	Casa

Client1

nclient	nom	ville
C 1	alami	Casa
C 4	anouari	Casa

Client2

nclient	nom	ville
C 2	saoudi	Rabat
C 3	saber	Fès

41

Architecture de la Donnée : La Fragmentation Horizontale

CONCEPT

Découpage de la relation en sous-ensembles de tuples (lignes).

[Partitionnement = Sélection (σ)]

[Recomposition = Union (U)]

EXÉCUTION

Client

nclient	nom	ville
C1	alami	Casa
C2	saoudi	Rabat
C3	saber	Fès
C4	anouari	Casa

Client1 (Casa)

nclient	nom	ville
C1	alami	Casa
C4	anouari	Casa

Client2 (Autres)

nclient	nom	ville
C2	saoudi	Rabat
C3	saber	Fès

Client1 = σ [ville = 'Casa'] Client

Client2 = σ [ville <> 'Casa'] Client

Client = Client1 U Client2

42

Fragmentation horizontale Exemple pratique

❖ Fragments définis par sélection

- Create table *Client1* as Select * From Client Where N_agence = 1
- Create table *Client2* as Select * From Client Where N_agence = 2
- Create table *Client3* as Select * From Client Where N_agence = 3

❖ Reconstruction de la relation globale

- Create view *Client* as Select * From *Client1* Union Select * From *Client2* Union Select * From *Client3*

43

Fragmentation horizontale dérivée

Fragments définis par semi-jointure

Create table *Cde1* as select * from Cde, Client1 where Cde.nclient = Client1.nclient

Create table *Cde2* as select * from Cde, Client2 where Cde.nclient = Client2.nclient

Reconstruction de la relation globale

Create view *Commande* as Select * From *Cde1* Union Select * From *Cde2*

44

FRAGMENTATION VERTICALE



Mécanisme :
Découpage par COLONNES
(Attributs).

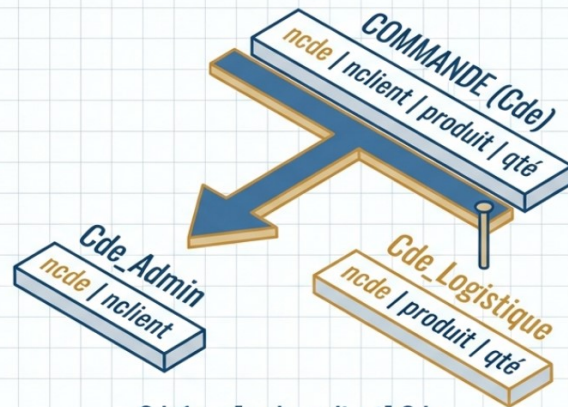


Opérateur :
Projection (π).



Reconstruction :
JOINTURE (JOIN).

Note : La Clé Primaire doit être
dupliquée dans tous les fragments.



$Cde1 = \pi[ncde, nclient] Cde$
 $Cde2 = \pi[ncde, produit, qté] Cde$

45

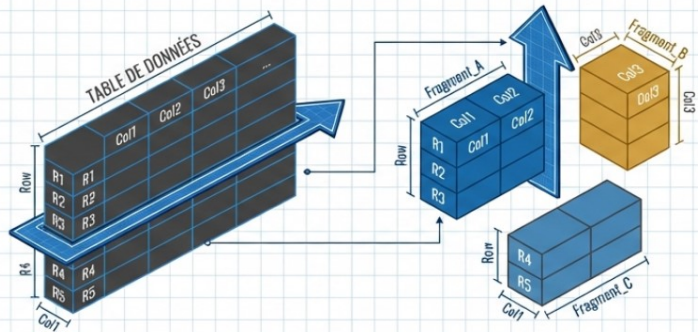
Fragmentation verticale

L'opérateur de partitionnement est la **projection**
(π)

L'opérateur de recomposition est la **jointure**

46

FRAGMENTATION MIXTE



Définition :

Combinaison de Sélections (σ) et de Projections (π).



Utilisation :

Pour des scénarios complexes où certaines lignes nécessitent moins de colonnes que d'autres.

$$\text{Reconstruction} = (\text{Fragment_A} \cup \text{Fragment_B}) * \text{Fragment_C}$$

(L'opération * représente la Jointure)

47

Architecture de la Donnée : La Fragmentation Verticale

CONCEPT

Découpage de la relation par attributs (colonnes).

Règle Cruciale : La clé primaire doit figurer dans tous les fragments pour permettre la jointure.

[Partitionnement = Projection (π)]

[Recomposition = Jointure ($\bowtie$)]

EXÉCUTION

ncde	nclient	produit	qté
D 1	C 1	P 1	10
D 2	C 1	P 2	20
D 3	C 2	P 3	5
D 4	C 4	P 4	10

ncde	nclient
D 1	C 1
D 2	C 1
D 3	C 2
D 4	C 4

ncde	produit	qté
D 1	P 1	10
D 2	P 2	20
D 3	P 3	5
D 4	P 4	10

$\text{Cde1} = \pi[\text{ncde}, \text{nclient}] \text{ Cde}$

$\text{Cde2} = \pi[\text{ncde}, \text{produit}, \text{qté}] \text{ Cde}$

48

Fragmentation verticale

Fragments définis par projection

- Cde1 = Cde (ncde, nclient)
- Cde2 = Cde (ncde, produit, qté)

ou

- Cde1 = π [ncde, nclient]Cde
- Cde2 = π [ncde, produit, qté]Cde

Reconstruction

- Cde = [ncde, nclient, produit, qté]
where Cde1.ncde = Cde2.ncde

Cde

ncde	nclient	produit	qté
D 1	C 1	P 1	10
D 2	C 1	P 2	20
D 3	C 2	P 3	5
D 4	C 4	P 4	10

Cde1

ncde	nclient
D 1	C 1
D 2	C 1
D 3	C 2
D 4	C 4

Cde2

ncde	produit	qté
D 1	P 1	10
D 2	P 2	20
D 3	P 3	5
D 4	P 4	10

49

Fragmentation verticale

Exemple pratique:

❖ Fragments définis par projection

- Create table *Compte1* as Select N_compte, Nom, Adresse, Solde From *Compte*
- Create table *Compte2* as Select N_compte, N_agence From *Compte*

❖ Reconstruction de la relation globale

- Select N_compte, N_agence , Nom, Adresse, Solde From *Compte1* , *Compte2* Where *Compte1*.N_compte=*Compte2*.N_compte

50

Fragmentation Mixte

Combinaison des fragmentations horizontales et verticales

L'opération de partitionnement est une combinaison de projections et de sélections.

L'opération de recomposition est une combinaison de jointures et d'unions.

51

Fragmentation Mixte

➤ L'opération de partitionnement

- Relation $cli3 \pi [NoClient, NomClient] (\sigma [Age < 38]Client)$
- Relation $cli5 \pi [NoClient, NomClient] (\sigma [Age \geq 38]Client)$
- Relation $cli4 \pi [NoClient, Prénom]Client$
- Relation $cli6 \pi [NoClient, Age]Client$

➤ L'opération de recomposition

- Client est obtenue avec : $(cli3 \cup cli5) * cli4 * cli6$

52

Stratégies Avancées : La Fragmentation Mixte

Une combinaison de projections (π) et de sélections (σ).

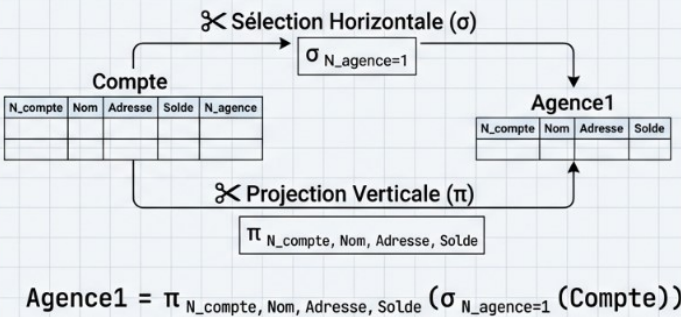
CONCEPT

Définition : Une combinaison de projections (π) et de sélections (σ).

Recomposition : Une combinaison de jointures ($\bowtie$) et d'unions ($\cup$).

[Partitionnement = Projection (π) et Sélection (σ)]
[Recomposition = Jointure ($\bowtie$) et Union ($\cup$)]

EXÉCUTION



Reconstruction :

- Client = (Agence1 $\bowtie$ N_compte Compte_agence) $\cup$
- (Agence2 $\bowtie$ N_compte Compte_agence) $\cup$
- (Agence3 $\bowtie$ N_compte Compte_agence)

53

Fragmentation Mixte

Agence1 = $\pi_{N_compte, Nom, Adresse, Solde}(\sigma_{N_agence=1}(Compte))$,
Agence2 = $\pi_{N_compte, Nom, Adresse, Solde}(\sigma_{N_agence=2}(Compte))$,
Agence3 = $\pi_{N_compte, Nom, Adresse, Solde}(\sigma_{N_agence=3}(Compte))$,
Compte_agence = $\pi_{N_compte, N_agence}(Compte)$

Reconstruction :

Client = (Agence1 $\bowtie$ N_compte Compte_agence) $\cup$
(Agence2 $\bowtie$ N_compte Compte_agence) $\cup$
(Agence3 $\bowtie$ N_compte Compte_agence)

Agence1			
N_compte	Nom	Adresse	Solde
1	Alami	1 Rue x	1000
2	El Bouanani	2 Rue y	1100

Agence2			
N_compte	Nom	Adresse	Solde
4	El Hachimi	1 Rue a	1300

Agence3			
N_compte	Nom	Adresse	Solde
3	Moutaouakil	1 Rue z	1200

Compte_agence	
N_compte	N_agence
1	1
2	1
3	2
4	3

54

Dans le contexte de la fragmentation horizontale, quel symbole de l'algèbre relationnelle représente l'opérateur de partitionnement ?

A. $\bowtie$

B. σ

C. π

D. $\cup$

55

Pour reconstruire la relation globale à partir de fragments horizontaux, quel opérateur est utilisé ?

A. Le produit cartésien

B. La jointure (*bowtie*)

C. La projection (*pi*)

D. L'union (*U*)

56

Quel opérateur définit le partitionnement dans une fragmentation verticale ?

- A. L'intersection
- B. La sélection (σ)
- C. La semi-jointure
- D. La projection (π)

57

Lors de la reconstruction d'une fragmentation verticale, quelle condition est indispensable pour retrouver la relation d'origine ?

- A. Effectuer une jointure sur la clé primaire
- B. Utiliser l'opérateur de sélection sur chaque fragment
- C. Effectuer une union de tous les fragments
- D. Vérifier que les fragments n'ont aucune colonne en commun

58

Dans la fragmentation mixte, comment est définie l'opération de recomposition ?

- A. Uniquement par des projections et des sélections
- B. Par une série de produits cartésiens
- C. Une combinaison de jointures et d'unions
- D. Uniquement par des unions successives

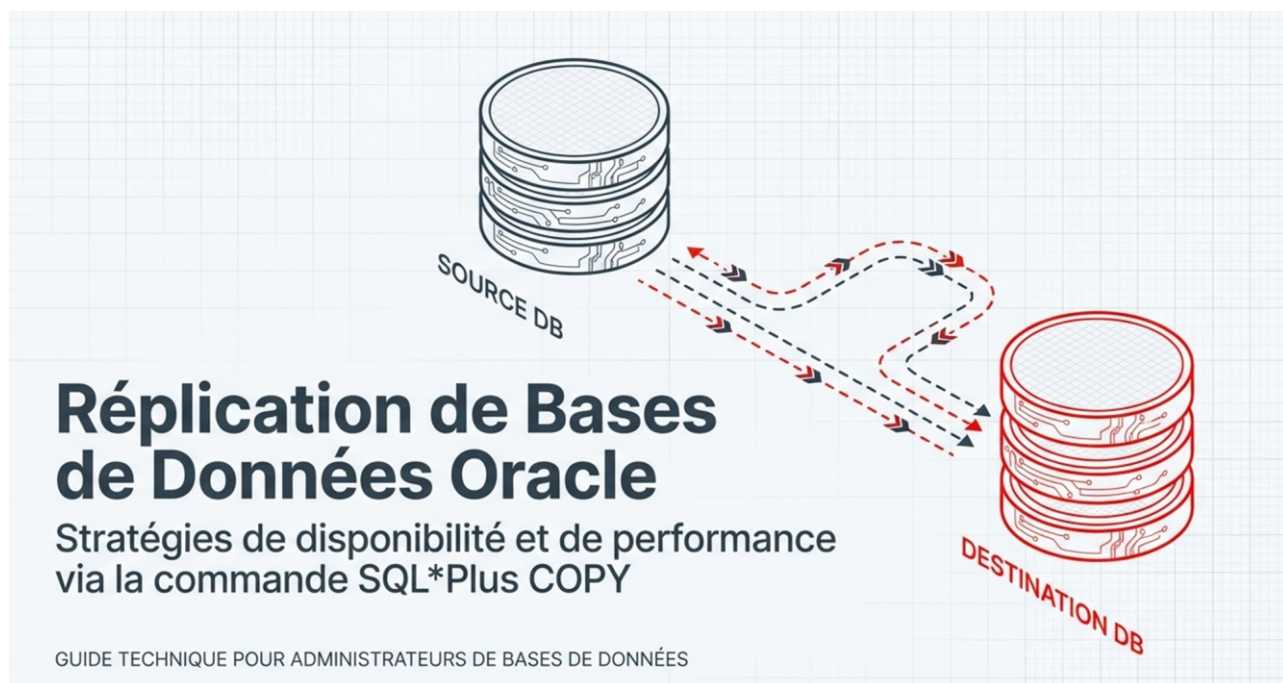
59

Communication entre deux sites

• Etapes de communication :

- 1- Communication entre les deux sites via les @IP.
- 2- Configuration listener pour les deux sites
- 3 -Activer les services Oracle:
 - Service SID
 - Service listener
- 4- Création des utilisateurs dans les deux sites
- 5- Accès à distance par les comptes users
- 6- Création des noms de services.
- 7- Accès à distances en utilisant les services name
- 8- Création d'un schéma global
 - Client(CodeCl, nom, age)
 - Produit(numPrd, marque,prix, #codeCl)

60



61

Réplication

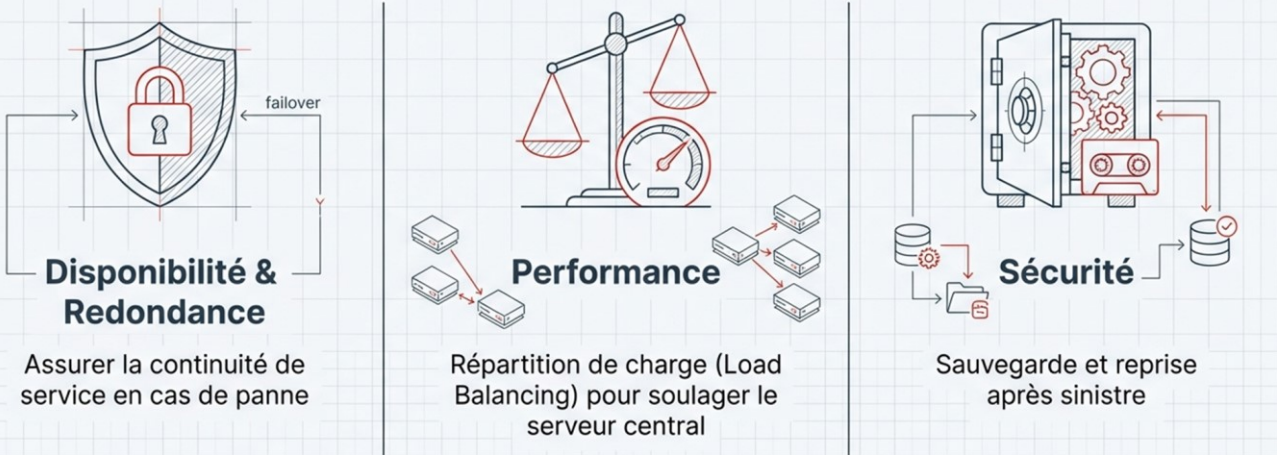
La réplication d'une base de données est le processus de **création de copies synchronisées** ou **asynchrones** d'une base de données afin **d'améliorer la disponibilité, la redondance et la performance.**

Cela peut être fait pour des raisons **de sauvegarde, de reprise après sinistre, de répartition de charge** ou **d'amélioration des performances globales du système.**

62

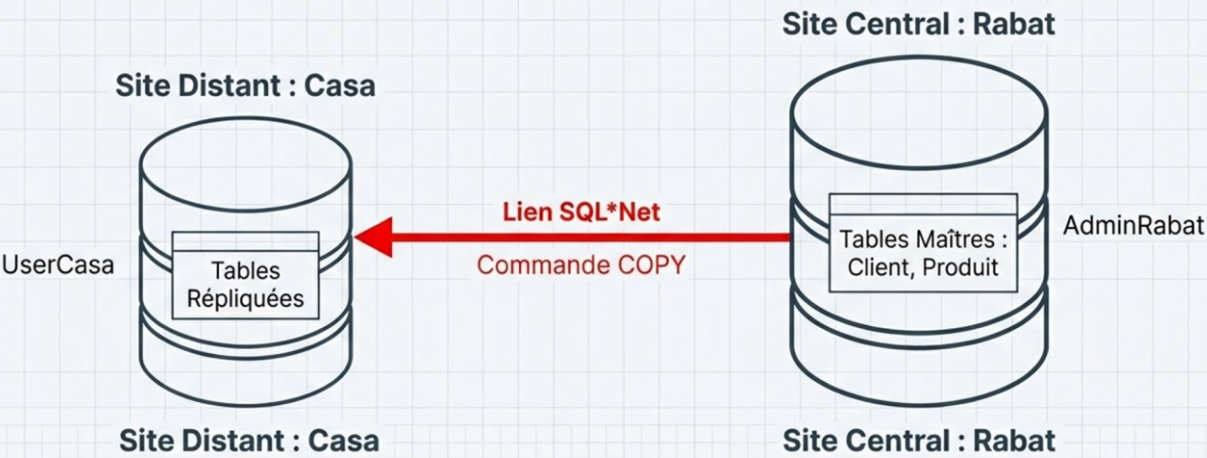
Pourquoi la Réplication ?

Processus de création de copies synchronisées ou asynchrones d'une base de données.



63

L'Architecture : Rabat vers Casa



Scénario : Transfert de données du siège (Rabat) vers une agence locale (Casa).

64

La Commande SQL*Plus COPY

```
COPY FROM BD_source TO BD_destination
```

```
{ ACTION }
```

```
table_destination [(colonne1, colonne2, ...)]
```

```
USING requête_fragmentation
```

Mode d'insertion
(Create, Insert, etc.)

Cette commande permet de répliquer des données directement via SQL*Plus sans outils d'exportation complexes.

65

Procédure de la réplication

➤ Commande COPY

La première option consiste à répliquer régulièrement les données sur le serveur local au moyen de la commande COPY de SQL*Plus.

```
COPY FROM BD_source TO BD_destination
```

```
{CREATE | INSERT | APPEND | REPLACE}
```

```
table_destination[(colonne1, colonne2, ...)]
```

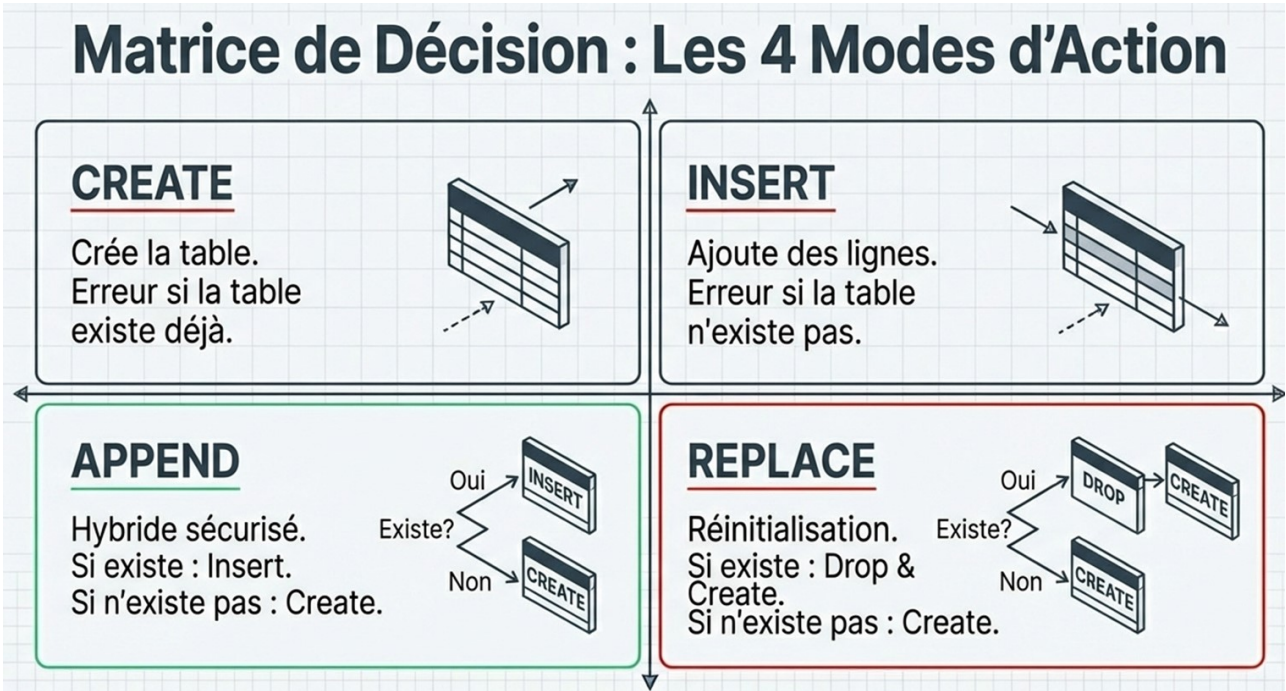
```
USING requête_fragmentation
```

66

Procédure de la réplication

- **CREATE** : Si la table destination existe, copy génère une erreur. Sinon, la table est créée et les données seront copiées : **CREATE + INSERT**
- **INSERT** : Si la table destination existe, copy ajoute les nouvelles lignes. Sinon, elle génère une erreur et s'arrête : **INSERT**
- **APPEND** : Si la table destination existe, copy ajoute les données . Sinon, elle crée la table et insert les données. : **[CREATE] + INSERT**
- **REPLACE** : Si la table destination existe, copy supprime et recrée la table avec les nouvelles données. Sinon, elle crée la table et insert les données : **[DROP] + CREATE + INSERT**

67



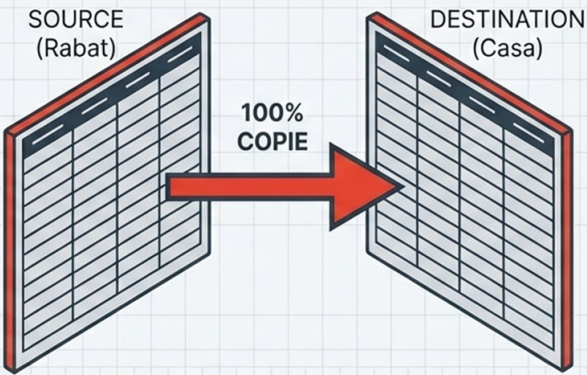
68

Stratégie 1 : Duplication Totale

Clonage intégral de la table Client.

```
COPY FROM UserRabat/UserRabat@Rabat
TO UserCasa/UserCasa@Casa
CREATE client
USING SELECT * FROM client;
```

La table destination devient une copie conforme de la source.

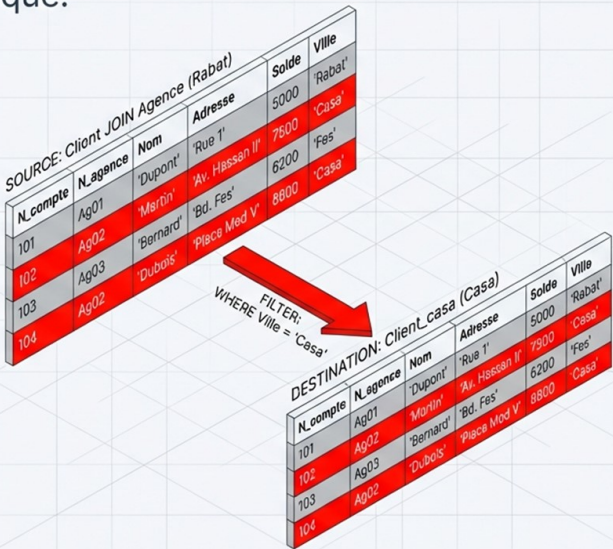


69

Stratégie 2 : Fragmentation Horizontale

Filtrer les lignes : Restriction géographique.

```
COPY FROM UserRabat/UserRabat@Rabat TO
UserCasa/UserCasa@Casa
CREATE Client_casa
USING SELECT N_compte, Client.N_agence,
Nom, Adresse, Solde FROM Client, Agence
WHERE Client.N_agence = Agence.N_agence
AND Ville='Casa';
```



70

Exemple de la réplication distante

Exemples :

Duplication

```
COPY FROM UserRabat/UserRabat@Rabat TO UserCasa/  
UserCasa@Casa  
CREATE client USING (Select N_client,nom_client, Ville  
FROM client )
```

Fragmentation horizontale

```
COPY FROM UserRabat/UserRabat@Rabat TO UserCasa/  
UserCasa@Casa CREATE Client_casa USING (Select N_compte,  
Client.N_agence, Nom, Adresse, Solde From Client, Agence  
Where Client.N_agence = Agence.N_agence and Ville='Casa' )
```

71

Stratégie 3 : Fragmentation Vertica

Filtrer les colonnes : Pertinence et Sécurité.

```
COPY FROM UserRabat/UserRabat@Rabat TO UserCasa/UserCasa@Casa  
APPEND Client_info  
USING SELECT N_compte, solde FROM Client;
```

SOURCE: Client

N_compte	Nom	Adresse	Solde	Ville
101	Dupont	Rue 1	5000	Rabat
102	Dupont	Rue 2	6000	Rabat
103	Dupont	Rue 3	2000	Rabat
104	Dupont	Rue 4	4000	Rabat
105	John	Rue 5	3000	Rabat
106	John	Rue 6	4000	Rabat
107	Marona	Rue 7	7000	Rabat
108	John	Rue 8	5000	Rabat
	Jannars	Rue 9	4000	Rabat
		Rue 10	2000	Ville
		Rue 11	2000	Ville
			5000	Ville

72

Exemple de la réplication distante

Exemples :

Fragmentation verticale (Ajout)

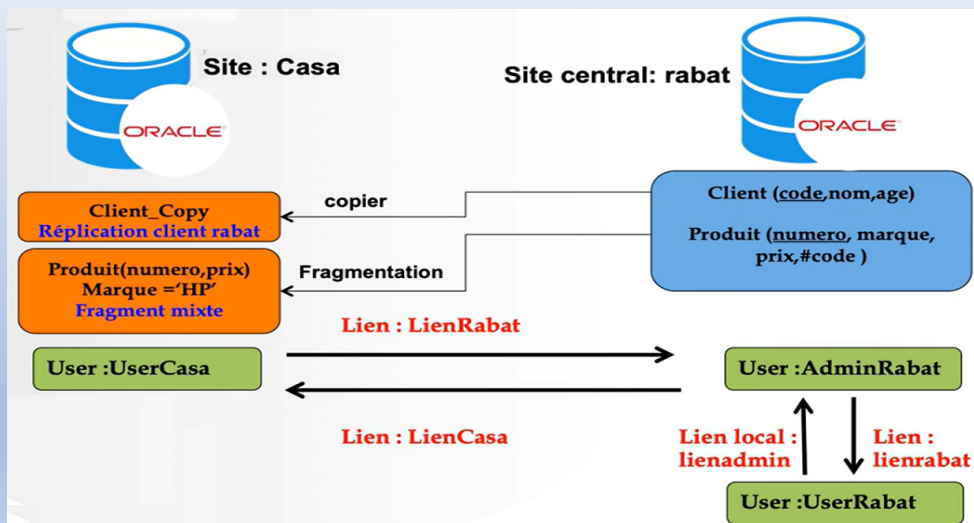
```
COPY FROM UserRabat/UserRabat@Rabat TO UserCasa/
UserCasa@Casa
APPEND Client_info USING (Select N_compte, solde From
Client )
```

Fragmentation verticale (Remplacement)

```
COPY FROM UserRabat/UserRabat@Rabat TO UserCasa/
UserCasa@Casa
REPLACE Client_info USING (Select N_compte, solde From
Client )
```

73

Schéma de la réplication



74

Stratégie 4 : Fragmentation Mixte

Ciblage précis : Lignes ET Colonnes.

```
COPY FROM user_rabat/user_rabat@local TO user_casa/user_casa@casa
CREATE produit USING SELECT numero, prix FROM produit
WHERE marque='hp';
```

SOURCE: produit

numero	marque	prix	code
33	DELL	4500	2
11	hp	3000	1
22	epson	2000	1

FILTER:
SELECT numero, prix
WHERE marque='hp'

DESTINATION: produit

numero	prix
11	3000

75

Exemple de la réplication sous oracle

```
SQL> select * from client;
CODE NOM          AGE
-----
4 sadki           23
1 Samahi          23
2 malki           42
3 fahmi           32

SQL> select * from produit;
NUMERO MARQUE      PRIX      CODE
-----
33 DELL           4500      2
11 hp             3000      1
22 epson          2000      1

SQL> connect userrabat/userrabat
ConnectU.
SQL> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa create client_copy using
select * from client;

Taille tableau extractions/variables est 15. (taille du tableau est 15)
Sera validé après opération. (COPYCOMMIT = 0)
Taille maximum (LONG) est 80. (longueur est 80)
La table CLIENT_COPY est créée.
```

76

Exemple de la réplication sous oracle

The screenshot shows two Oracle SQL*Plus sessions. The left session displays the source data from the 'client' table, and the right session displays the replicated data in the 'client_copy' table.

Source Data (client table):

CODE	NOM
4	sadki
1	Samahi
2	maliki
3	fahmi

Replicated Data (client_copy table):

CODE	NOM	AGE
4	sadki	23
1	Samahi	23
2	maliki	42
3	fahmi	32

77

Exemple de la réplication sous oracle

```
SQL> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa create client_copy using
select * from client;

Taille tableau extractions/variables est 15. (taille du tableau est 15)
Sera validé après opération. (COPYCOMMIT = 0)
Taille maximum (LONG) est 80. (longueur est 80)

ERROR:
ORA-00955: ce nom d'objet existe déjà

SQL> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa insert client_copy using
select * from client;

Taille tableau extractions/variables est 15. (taille du tableau est 15)
Sera validé après opération. (COPYCOMMIT = 0)
Taille maximum (LONG) est 80. (longueur est 80)
4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.

SQL>
```

78

Exemple de la réplication sous oracle

```
pat/adminrab SQL> select* from client_copy;
CODE NOM AGE
-----
4 sadki 23
1 Samahi 23
2 malki 42
3 fahmi 32

pat/adminrab SQL> select* from client_copy;
CODE NOM AGE
-----
4 sadki 23
1 Samahi 23
2 malki 42
3 fahmi 32
4 sadki 23
1 Samahi 23
2 malki 42
3 fahmi 32

8 ligne(s) sùlectionnÙe(s).
SQL>
```

79

Exemple de la réplication sous oracle

```
4 sadki 23
1 Samahi 23
2 malki 42
3 fahmi 32

SQL> select* from client_copy;
CODE NOM AGE
-----
4 sadki 23
1 Samahi 23
2 malki 42
3 fahmi 32
4 sadki 23
1 Samahi 23
2 malki 42
3 fahmi 32

8 ligne(s) sùlectionnÙe(s).
SQL> drop table client_copy;
Table supprimÙe.
SQL>
```

80

Exemple de la réplcation sous oracle

```
4 lignes validées en CLIENT_COPY à usercasa@casa.
> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa insert client_copy using
lect * from client;

lle tableau extractions/variables est 15. (taille du tableau est 15)
a validé aprs opération. (COPYCOMMIT = 0)
lle maximum (LONG) est 80. (longueur est 80)
OR:
-00942: Table ou vue inexistante

> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa append client_copy using
lect * from client;

lle tableau extractions/variables est 15. (taille du tableau est 15)
a validé aprs opération. (COPYCOMMIT = 0)
lle maximum (LONG) est 80. (longueur est 80)
table CLIENT_COPY est créée.

4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.
4 lignes validées en CLIENT_COPY à usercasa@casa.

> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa append client_copy using
lect * from client;

lle tableau extractions/variables est 15. (taille du tableau est 15)
a validé aprs opération. (COPYCOMMIT = 0)
lle maximum (LONG) est 80. (longueur est 80)
4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.
```

81

Exemple de la réplcation sous oracle

```
> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa replace client_copy using
lect * from client;

lle tableau extractions/variables est 15. (taille du tableau est 15)
a validé aprs opération. (COPYCOMMIT = 0)
lle maximum (LONG) est 80. (longueur est 80)
table CLIENT_COPY est supprimée.
table CLIENT_COPY est créée.
```

SQL> select* from client_copy;

CODE	NOM	AGE
4	sadki	23
1	Samahi	23
2	maiki	42
3	fahmi	32
4	sadki	23
1	Samahi	23
2	maiki	42
3	fahmi	32

8 ligne(s) sélectionnée(s).

SQL> select* from client_copy;

CODE	NOM	AGE
4	sadki	23
1	Samahi	23
2	maiki	42
3	fahmi	32

SQL>

82

Exemple de la réplication sous oracle

```
> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa replace client_copy using
select * from client;

11e tableau extractions/variables est 15. (taille du tableau est 15)
a validé après opération. (COPYCOMMIT = 0)
11e maximum (LONG) est 80. (longueur est 80)
table CLIENT_COPY est supprimée.
table CLIENT_COPY est créée.

4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.

> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa replace client_copy using
select * from client;

11e tableau extractions/variables est 15. (taille du tableau est 15)
a validé après opération. (COPYCOMMIT = 0)
11e maximum (LONG) est 80. (longueur est 80)
table CLIENT_COPY est supprimée.
table CLIENT_COPY est créée.

4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.
```

```
SQL> select * from client_copy;
-----
CODE NOM                AGE
-----
4 sadki                  23
1 Samahi                 42
2 maliki                 32
3 fahmi                  32

SQL> select * from client_copy;
-----
CODE NOM                AGE
-----
4 sadki                  23
1 Samahi                 42
2 maliki                 32
3 fahmi                  32

SQL>
```

83

Exemple de la réplication sous oracle

```
4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.

> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa append client_copy using
select * from client;

11e tableau extractions/variables est 15. (taille du tableau est 15)
a validé après opération. (COPYCOMMIT = 0)
11e maximum (LONG) est 80. (longueur est 80)
4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.

> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa
select * from client;

11e tableau extractions/variables est 15. (taille du tableau est 15)
a validé après opération. (COPYCOMMIT = 0)
11e maximum (LONG) est 80. (longueur est 80)
table CLIENT_COPY est supprimée.
table CLIENT_COPY est créée.

4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.

> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa
select * from client;

11e tableau extractions/variables est 15. (taille du tableau est 15)
a validé après opération. (COPYCOMMIT = 0)
11e maximum (LONG) est 80. (longueur est 80)
table CLIENT_COPY est supprimée.
table CLIENT_COPY est créée.

4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.
```

```
SQL> select * from client_copy;
-----
CODE NOM                AGE
-----
4 sadki                  23
1 Samahi                 42
2 maliki                 32
3 fahmi                  32

SQL> select * from client_copy;
-----
CODE NOM                AGE
-----
4 sadki                  23
1 Samahi                 42
2 maliki                 32
3 fahmi                  32

SQL> drop table client_copy;
Table supprimée.

SQL>
```

84

Exemple de la réplication sous oracle

```
> copy from adminrabat/adminrabat@local to usercasa/usercasa
select * from client;

1 le tableau extractions/variables est 15. (taille du tableau)
2 a validé après opération. (COPYCOMMIT = 0)
3 le maximum (LONG) est 80. (longueur est 80)
4 table CLIENT_COPY est supprimée.
5 table CLIENT_COPY est créée.
6 4 lignes sélectionnées à partir de adminrabat@local.
7 4 lignes insérées dans CLIENT_COPY.
8 4 lignes validées en CLIENT_COPY à usercasa@casa.

> copy from adminrabat/adminrabat@local to usercasa/usercasa
select * from client;

1 le tableau extractions/variables est 15. (taille du tableau)
2 a validé après opération. (COPYCOMMIT = 0)
3 le maximum (LONG) est 80. (longueur est 80)
4 table CLIENT_COPY est supprimée.
5 table CLIENT_COPY est créée.
6 4 lignes sélectionnées à partir de adminrabat@local.
7 4 lignes insérées dans CLIENT_COPY.
8 4 lignes validées en CLIENT_COPY à usercasa@casa.

>
```

```
CODE NOM
-----
4 sadki
1 Samahi
2 malki
3 fahmi

SQL> drop table client_copy;
Table supprimée.

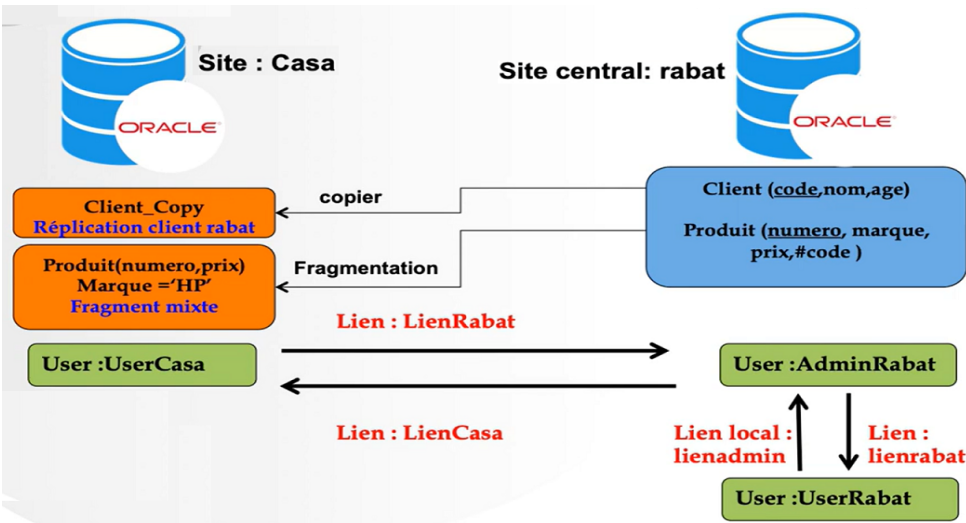
SQL> select * from client_copy;

CODE NOM
-----
4 sadki
1 Samahi
2 malki
3 fahmi

SQL>
```

85

Schéma de la réplication



86

exemple : Fragmentation:

```
> copy from adminrabat/adminrabat@local to usercasa/usercasa@casa create produit using sel
numero,prix from produit where marque='hp';

lle tableau extractions/variables est 15. (taille du tableau est 15)
a validé après opération. (COPYCOMMIT = 0)
lle maximum (LONG) est 80. (longueur est 80)
table PRODUIT est créée.

1 lignes sélectionnées à partir de adminrabat@local.
1 lignes insérées dans PRODUIT.
1 lignes validées en PRODUIT à usercasa@casa.

> show user
R est "USERRABAT"
> connect adminrabat/adminrabat
necté.
> select * from produit;
```

NUMERO	MARQUE	PRIX	CODE
33	DELL	4500	2
11	hp	3000	1
22	epson	2000	1

```
>
```

87

Résultat de la fragmentation Mixte

```
> copy from adminrabat/adminrabat@local to
select * from client;

lle tableau extractions/variables est 15.
a validé après opération. (COPYCOMMIT = 0)
lle maximum (LONG) est 80. (longueur est 80)
table CLIENT_COPY est créée.

4 lignes sélectionnées à partir de adminra
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercas

> copy from adminrabat/adminrabat@local to
numero,prix from produit where marque='hp'

lle tableau extractions/variables est 15.
a validé après opération. (COPYCOMMIT = 0)
lle maximum (LONG) est 80. (longueur est 80)
table PRODUIT est créée.

1 lignes sélectionnées à partir de adminra
1 lignes insérées dans PRODUIT.
1 lignes validées en PRODUIT à usercasa@ca

> show user
R est "USERRABAT"
> connect adminrabat/adminrabat
necté.
> select * from produit;
```

NUMERO	MARQUE	PRIX	CODE
33	DELL	4500	2
11	hp	3000	1
22	epson	2000	1

```
>
```

```
C:\Windows\system32\cmd.exe - sqlplus

4 sadki 23
1 Samahi 23
2 malki 42
3 fahmi 32

SQL> drop table client_copy;
Table supprimée.

SQL> select * from client_copy;

CODE NOM AGE
-----
4 sadki 23
1 Samahi 23
2 malki 42
3 fahmi 32

SQL> select * from produit;

NUMERO PRIX
-----
11 3000

SQL>
```

88

Analyse d'Impact : Append vs Replace

Scenario A: APPEND

```
> copy from adminrabat/adminrabat@local to
usercasa/usercasa@casa append client_copy using select
* from client;
...
4 lignes sélectionnées à partir de adminrabat@local.
4 lignes insérées dans CLIENT_COPY.
4 lignes validées en CLIENT_COPY à usercasa@casa.
```

Mise à jour incrémentale (Ajout de données).

Scenario B: REPLACE

```
> copy from adminrabat/adminrabat@local to
usercasa/usercasa@casa replace client_copy using select
* from client;
...
Table CLIENT_COPY est supprimée.
Table CLIENT_COPY est créée.
```

Mise à jour complète (Refresh destructif).

Gestion des Erreurs Courantes

⚠️ **ORA-00955** : Ce nom d'objet existe déjà

Cause : Utilisation de CREATE sur une table existante.
Solution : Passer en mode APPEND ou REPLACE.

⚠️ **ORA-00942** : Table ou vue inexistante

Cause : Utilisation de INSERT vers une table non créée.
Solution : Passer en mode CREATE.

Synthèse des Bonnes Pratiques



Planifier

Définir les besoins
(Rabat vs Casa).



Choisir

Stratégie Horizontale,
Verticale, ou Mixte ?



Exécuter

APPEND (Sécurité) ou
REPLACE (Fraîcheur).

```
COPY FROM [Source] TO [Dest] [Action] [Table] USING [Query]
```

La réplication efficace équilibre la fraîcheur des données et la charge réseau.